

LeetCode 317 — Shortest Distance from All Buildings (Premium)

Authored by: Arpit Raj, LNMIIT Jaipur | July 27, 2026

Problem Statement

You are given a grid:

- **0** → empty land
- **1** → building
- **2** → obstacle

You need to find an empty land cell (0) such that the sum of shortest distances from all buildings to this cell is minimum. If no such cell exists, return -1.

Example Grid

B	.	#	.	B
.	.	.	.	.
.	.	B	.	.

Buildings: (0,0), (0,4), (2,2)

We try every empty cell and calculate:

distance from building 1 + distance from building 2 + distance from building 3

The answer is the cell with the minimum total distance.

First Thought

Can we run BFS from every empty cell?

Suppose: $n * m = 10000$

Empty cells can be ~ 10000 . Each BFS: $O(n * m)$

Total: $O((n * m)^2)$ — **Too slow.**

Optimal Approach

Number of buildings is usually much smaller. So reverse the thinking.

Instead of:

Empty cell → BFS to all buildings

do:

Building → BFS to all empty cells

Maintain Two Matrices

1. distance matrix

`dist[r][c]` stores total distance from all buildings.

Example after processing first building:

0 1 2

1 2 3

2 3 4

After second building:

5 3 4

4 4 5

3 4 6

2. reach count matrix

`reach[r][c]` stores how many buildings can reach this cell.

Why?

Because a cell is valid only if:

number of buildings reaching
this cell = total buildings

A cell blocked by an obstacle may not
receive BFS from some buildings.

Algorithm Execution

For every building: Run BFS. During BFS:

- Ignore obstacles.
- Ignore other buildings.
- Visit empty cells.

For every empty cell reached:

```
dist[nr][nc] += current_distance;  
reach[nr][nc]++;
```

After all buildings are processed, traverse the grid to find a cell where:

```
grid[i][j] == 0 AND reach[i][j] == number_of_buildings
```

Answer: `min(dist[i][j])`


```

class Solution {
public:
    vector<pair<int, int>> dir = {{-1,0}, {1,0}, {0,-1}, {0,1}};

    void bfs(int sr, int sc, vector<vector<int>>& grid,
            vector<vector<int>>& dist, vector<vector<int>>& reach) {

        int n = grid.size();
        int m = grid[0].size();

        queue<pair<int, int>> q;
        vector<vector<int>> seen(n, vector<int>(m, 0));

        q.push({sr, sc});
        seen[sr][sc] = 1;

        int level = 0;

        while(!q.empty()) {
            int sz = q.size();
            level++;

            while(sz--) {
                int r = q.front().first;
                int c = q.front().second;
                q.pop();

                for(auto &it : dir) {
                    int nr = r + it.first;
                    int nc = c + it.second;

                    if(nr >= 0 && nc >= 0 && nr < n && nc < m &&
                       !seen[nr][nc] && grid[nr][nc] == 0) {

                        seen[nr][nc] = 1;

                        // this empty cell is reachable from current building
                        dist[nr][nc] += level;

                        // one more building can reach this cell
                        reach[nr][nc]++;

                        q.push({nr, nc});
                    }
                }
            }
        }
    }

    int shortestDistance(vector<vector<int>>& grid) {
        int n = grid.size();
        int m = grid[0].size();

        vector<vector<int>> dist(n, vector<int>(m, 0));
        // number of buildings which can reach this cell
        vector<vector<int>> reach(n, vector<int>(m, 0));
    }
}

```

```

int buildings = 0;

// run BFS from every building
for(int i = 0; i < n; i++) {
    for(int j = 0; j < m; j++) {
        if(grid[i][j] == 1) {
            buildings++;
            bfs(i, j, grid, dist, reach);
        }
    }
}

int ans = INT_MAX;

// find empty land reachable from all buildings
for(int i = 0; i < n; i++) {
    for(int j = 0; j < m; j++) {
        if(grid[i][j] == 0 && reach[i][j] == buildings) {
            ans = min(ans, dist[i][j]);
        }
    }
}

if(ans == INT_MAX) {
    return -1;
}

return ans;
}
};

```